Android Architecture Patterns

MVP · MVVM · MVI · Clean · Multi-module · KMP · Voyager / Decompose / Circuit

Diagrams | Sample code | SOLID examples | Where-things-fit matrix

About this sheet: a personal high-density study aid covering every architecture pattern that matters in modern Android. Each pattern gets the same treatment: *why it exists, the diagram, pros/cons, sample code (View → ViewModel/Presenter → Repo → Data), where to put services/WorkManager/ContentProvider/Room/DI, KMP-friendly version, and the common mistakes*. Plus SOLID with concrete Android examples for "why this not that" interview questions, package structure templates, and how to start a fresh project. Written in plain English with original code examples.

1. Why architecture matters — the problem we're solving

Without architecture, all your code lives in one place. In early Android (2010-ish), that one place was the Activity. The "god Activity" problem looked like this:

```
class ProfileActivity : Activity() {
    override fun onCreate(b: Bundle?) {
        super.onCreate(b)
        setContentView(R.layout.profile)

        val retrofit = Retrofit.Builder()...build()        // Network setup
        val db = openOrCreateDatabase(...)               // DB access

        Thread {                                          // Threading
            val raw = retrofit.create(Api::class.java).fetchUser(42).execute()
            val user = parseAndValidate(raw)               // Business logic
            db.execSQL("INSERT...")                       // Persistence
            runOnUiThread {
                findViewById<TextView>(R.id.name).text = user.name   // UI
                findViewById<TextView>(R.id.email).text = user.email
            }
        }.start()
    }
}
```

Problems with this code:

- **Cannot test** — Activity needs Android framework; can't run on JVM.
- **Cannot reuse** — fetch logic is tied to this Activity.

- **Lifecycle bugs** — rotation kills the Thread but the network response still tries to update destroyed views.
- **Single Responsibility violation** — one class doing networking, persistence, threading, UI.
- **Hard to evolve** — change one piece and four others break.

Architecture patterns are answers to these problems. They impose a **separation of concerns**: each piece of code does one thing well, and the pieces communicate through well-defined contracts. This makes code testable, reusable, and survivable.

2. The shared mental model — three layers + UDF

Every modern Android architecture pattern is a refinement of the same three-layer idea inherited from broader software design (Hexagonal Architecture, Clean Architecture, Domain-Driven Design):

```
┌──────────────────────────────────────────────────┐
│  PRESENTATION LAYER                              │
│  (View, ViewModel/Presenter, UI state)           │
│  Knows about Android framework                   │
└──────────────────────────────────────────────────┘
                  │
                  │ depends on
                  ▼
┌──────────────────────────────────────────────────┐
│  DOMAIN LAYER (optional but recommended for big) │
│  (UseCases, business rules, domain models)       │
│  PURE Kotlin — no Android, no Retrofit, no Room  │
└──────────────────────────────────────────────────┘
                  │
                  │ depends on
                  ▼
┌──────────────────────────────────────────────────┐
│  DATA LAYER                                      │
│  (Repository, DataSources: Room, Retrofit, DataStore) │
│  Knows about persistence/network details         │
└──────────────────────────────────────────────────┘
```

Two universal rules:

1. **Dependencies point inward.** UI depends on Domain; Domain depends on nothing. Data also depends on Domain (because it implements Domain interfaces). This is the *Dependency Inversion Principle* in action.
2. **Unidirectional Data Flow (UDF).** Events go up (UI → ViewModel → Repo); state comes down (Repo → ViewModel → UI). No state mutates "sideways" between siblings.

What changes between MVP, MVVM, and MVI is **how the presentation layer is structured** — not the data layer. Most "MVVM vs MVI" debates are really debates about state management, not full architecture.

3. SOLID with concrete Android examples (interview-ready)

Five principles from Robert C. Martin. Most Android architecture choices (Repository, Hilt, ViewModel) are SOLID applied. Knowing concrete examples lets you defend any design decision in an interview.

S — Single Responsibility Principle

"A class has one reason to change."

VIOLATES SRP

```
class UserManager {
    fun fetchUser() = api.user()
    fun saveUser() = db.insert()
    fun showUser() = view.render()
    fun validateUser() = ...
}
```

One class changes for: API change, DB schema change, UI change, validation rule change.

FOLLOWS SRP

```
class UserApi { fun fetch() = ... }
class UserDao { fun save() = ... }
class UserRepository(api, dao)
class UserViewModel(repo)
class UserValidator()
```

Each class has exactly one reason to change.

O — Open/Closed Principle

"Open for extension, closed for modification." Add new behavior without touching existing code.

```
// Bad — switching on type, must edit every time you add a payment method
fun charge(method: String, amount: Long) = when (method) {
    "card" -> chargeCard(amount)
    "upi"  -> chargeUpi(amount)
    "wallet" -> chargeWallet(amount)
}

// Good — extend via interface, no modification needed for new types
interface PaymentMethod { suspend fun charge(amount: Long) }
class CardPayment : PaymentMethod { ... }
class UpiPayment : PaymentMethod { ... }

class Checkout(val method: PaymentMethod) {
    suspend fun charge(amount: Long) = method.charge(amount)
}
```

Android examples: OkHttp Interceptors (add new without editing OkHttpClient), Compose Modifiers, Hilt multibinding (`@IntoSet`).

L — Liskov Substitution Principle

"Subtypes must be drop-in replacements for their parent type — without changing program correctness."

```
// Production
class UserRepositoryImpl(val api: UserApi) : UserRepository {
    override suspend fun getUser(id: String) = api.fetch(id)
}

// In tests — must satisfy the SAME contract
class FakeUserRepository(val users: Map<String, User>) : UserRepository {
    override suspend fun getUser(id: String) = users[id] ?: error("not found")
}

// ViewModel doesn't know which one it got — that's the LSP guarantee
class UserVm(val repo: UserRepository) { ... }
```

Common violation: a subclass throws `UnsupportedOperationException` for a method the parent declares — breaks substitutability.

I — Interface Segregation Principle

"Many small focused interfaces beat one fat interface."

```
// Bad — fat interface forces every consumer to know all of it
interface Repository {
    suspend fun getUsers(): List<User>
    suspend fun getOrders(): List<Order>
    suspend fun getNotifications(): List<Notif>
    suspend fun getReports(): List<Report>
}

// Good — focused interfaces, easier to fake/test/replace
interface UserRepository  { suspend fun getUsers(): List<User> }
interface OrderRepository { suspend fun getOrders(): List<Order> }
interface NotifRepository { suspend fun getNotifications(): List<Notif> }
```

D — Dependency Inversion Principle

"Depend on abstractions, not on concretions. High-level modules don't know about low-level details."

```
// Bad — ViewModel knows about Retrofit, Room, network details
class UserVm {
    private val retrofit = Retrofit.Builder()...build()
    private val api = retrofit.create(UserApi::class.java)
    suspend fun load() = api.user()                    // tightly coupled
}

// Good — depend on abstraction; let DI inject the impl
class UserVm @Inject constructor(val repo: UserRepository) {
    suspend fun load() = repo.getUser()
}
// repo is an interface; implementation is in data layer
// Hilt module wires UserRepositoryImpl in production, FakeUserRepository in tests
```

This single principle is why Android architecture works at all. Every "this layer doesn't know about that layer" rule descends from DIP.

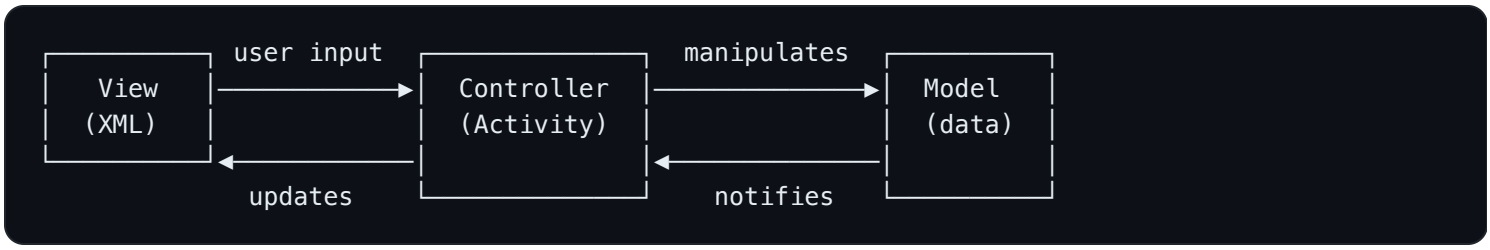
4. The history — how we got here

ERA	PATTERN	WHAT PROBLEM IT SOLVED
~2008-2012	God Activity	(it didn't — it WAS the problem)
~2012-2014	MVC (loose)	"Move some logic out of the Activity into a Controller-ish class"
~2014-2017	MVP	Testable presentation layer; Presenter is plain Kotlin/Java, View is a passive interface
~2017-2020	MVVM (Jetpack)	Eliminated Presenter↔View boilerplate via observable state (LiveData → Flow)
~2020+	MVI	Tame complex async screens with a single immutable state + reducer
~2018+	Clean Architecture (UseCases)	Add a Domain layer between UI and Data for shared business logic across screens or platforms
~2020+	Multi-module	Faster builds, enforced boundaries, dynamic delivery
~2022+	KMP-shared modules	Share business logic between Android + iOS + web
2023+	Voyager / Decompose / Circuit	Compose-first navigation + state management with stronger guarantees

Notice: each new pattern is a refinement, not a replacement. Many production apps use a combination — e.g., MVVM at the screen level, Clean Architecture for Domain, multi-module at the project level, and KMP for the data layer.

5. MVC — the brief context

"Model-View-Controller" came from Smalltalk-80 in the 1970s. Used everywhere from Rails to iOS. In Android, "MVC" was never a clean implementation — the Activity was both View and Controller, leading to god-class problems.



Why MVC failed in Android:

- The Controller (Activity) was tied to the framework — couldn't unit test.
- View (XML) had no logic, but the Activity rendered into the View directly via `findViewById` — making it a coupled View+Controller hybrid.
- No clear contract between View and Controller.

For new code: skip MVC. Mentioned only because some interview questions ask "why did Android move past MVC?"

6. MVP — Model-View-Presenter

Diagram

```
┌──────────────────┐                 ┌──────────────────┐                 ┌──────────────────┐
│       VIEW       │ user action     │    PRESENTER     │  fetch / save   │       MODEL      │
│  Activity /      │────────────────▶│  Plain Kotlin    │────────────────▶│  Repository +    │
│  Fragment        │                 │  class — no      │                 │  DataSources     │
│  (passive)       │                 │  Android refs    │                 │                  │
└──────────────────┘                 └──────────────────┘                 └──────────────────┘
         │                                    │                                    │
         ▲                                    │                                    │
         │  presenter.attach(view)            │                                    │
         │  view.showUser(user)               │  view.showUser(user)               │
         └──── interface contract ────────────┴────────────────────────────────────┘
                  (View interface)
```

Origin / inspiration

Came from **Martin Fowler's article on UI patterns** (Presentation Model). The thought process: "MVC is broken in fat-client UIs because the Controller can't easily test the View. So let's flip it — make the View dumb (passive interface), and put the View-driving logic in a separate Presenter class that only talks to the View through that interface. Now we can fake the View in tests."

Sample code

```
// 1. View interface — what the Presenter knows about the screen
interface UserView {
    fun showLoading()
    fun showUser(user: User)
    fun showError(message: String)
}

// 2. Presenter — pure Kotlin, testable on JVM
class UserPresenter(
    private val view: UserView,
    private val repo: UserRepository,
    private val scope: CoroutineScope
) {
    fun loadUser(id: String) {
        view.showLoading()
        scope.launch {
            try { view.showUser(repo.getUser(id)) }
            catch (e: Exception) { view.showError(e.message ?: "error") }
        }
    }

    fun detach() { scope.cancel() }
}

// 3. Activity — implements View; forwards events to Presenter
class UserActivity : AppCompatActivity(), UserView {
    private lateinit var presenter: UserPresenter

    override fun onCreate(b: Bundle?) {
        super.onCreate(b)
        setContentView(R.layout.user)
        presenter = UserPresenter(this, AppContainer.userRepo, MainScope())
        presenter.loadUser(intent.getStringExtra("id")!!)
    }

    override fun showLoading() { findViewById<...>(R.id.spinner).isVisible = true }
    override fun showUser(user: User) { ... }
    override fun showError(message: String) { ... }

    override fun onDestroy() { presenter.detach(); super.onDestroy() }
}

// 4. Repository — same as MVVM/MVI, see §7
```

Pros and cons

PROS

- Fully testable Presenter (no Android dependencies)
- Clear contract via View interface
- Works with Java codebases (no Coroutines required)
- Easy to understand — explicit method calls

CONS

- Lots of interface boilerplate per screen
- Presenter holds View reference → leak risk if not detached on destroy
- Doesn't survive configuration changes — must re-attach
- State management is manual (Presenter mutates fields)
- Doesn't fit reactive UIs naturally — no observable streams

Common mistakes

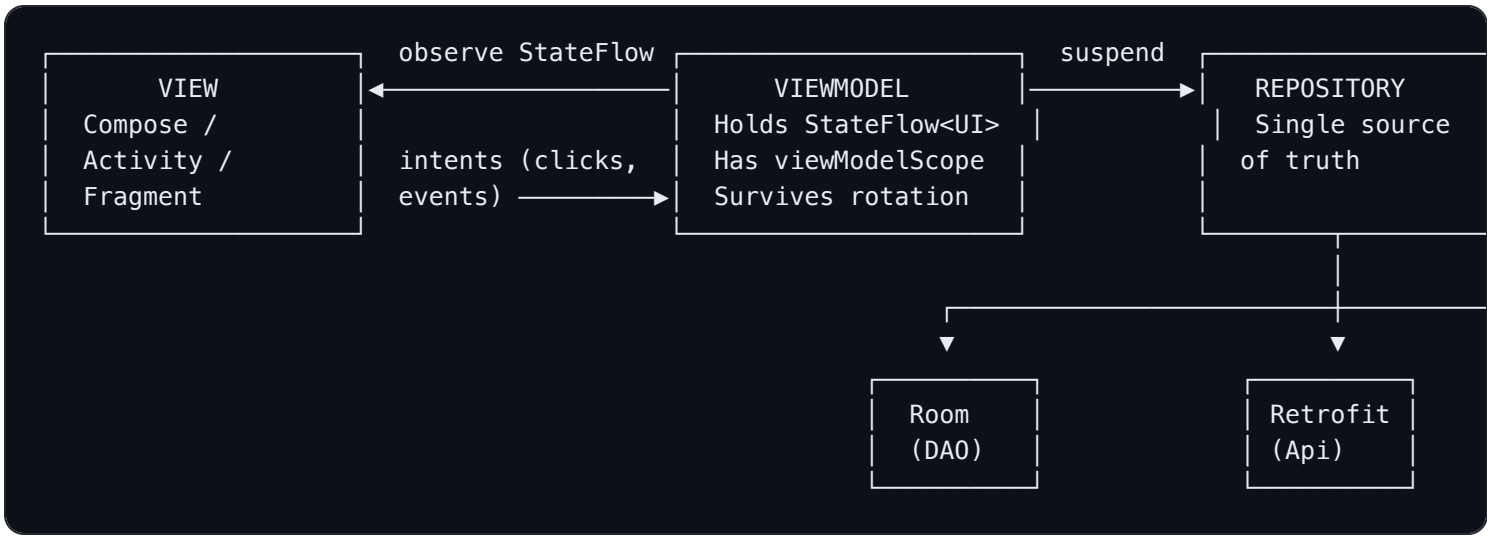
- **Forgetting to detach** the Presenter in `onDestroy()` → memory leak (Presenter holds Activity).
- **Presenter calling Android APIs** directly — defeats the testability purpose.
- **Bloated View interface** with 30 methods — split per screen state instead.
- **Storing UI state in the Presenter** — gets lost on rotation. Use a Bundle or move to MVVM.

Where things fit in MVP

COMPONENT	WHERE
Activity / Fragment	View layer — implements View interface, no business logic
Presenter	Presentation layer — orchestrates View ↔ Model
Repository	Data layer — single source of truth
Retrofit / Room / DataStore	Inside Repository — data sources
WorkManager	Triggered from Repository, not from Presenter (Presenter doesn't know about background)
ContentProvider	Wrap inside a DataSource class; expose suspend functions
Service / ForegroundService	Started from Repository or Application; communicates back via Repository's Flow
DI (Koin/Hilt/Manual)	Inject Presenter into Activity (manual) or via Hilt's `@AndroidEntryPoint`

7. MVVM — Model-View-ViewModel (the Android default)

Diagram



Origin / inspiration

MVVM was created by Microsoft for WPF/Silverlight (2005). The thought process: "Presenter ↔ View becomes painful when you want a richer View. What if the View just *observes* data exposed by the ViewModel, and changes happen by updating the data — not by calling methods on the View?" That observation pattern is straight out of **the Observer pattern** from Gang of Four; Android's LiveData and Flow are concrete implementations.

Android picked it up around 2017 with **Architecture Components (ViewModel + LiveData)**. The framework `ViewModel` class survives configuration changes — a problem that MVP couldn't easily solve.

Sample code (View → ViewModel → Repository → Data)

```kotlin
// 1. UI state — single immutable representation of the screen
data class UserUiState(
    val loading: Boolean = false,
    val user: User? = null,
    val error: String? = null
)

// 2. ViewModel — exposes state, accepts events
@HiltViewModel
class UserViewModel @Inject constructor(
    private val repo: UserRepository,
    private val handle: SavedStateHandle
) : ViewModel() {
    private val _uiState = MutableStateFlow(UserUiState())
    val uiState: StateFlow<UserUiState> = _uiState.asStateFlow()

    init { load(handle.get<String>("id") ?: return) }

    fun load(id: String) {
        viewModelScope.launch {
            _uiState.update { it.copy(loading = true, error = null) }
            try {
                val user = repo.getUser(id)
                _uiState.update { it.copy(loading = false, user = user) }
            } catch (e: Exception) {
                _uiState.update { it.copy(loading = false, error = e.message) }
            }
        }
    }

    fun refresh() = load(handle.get<String>("id")!!)
}

// 3. View (Compose) — observes state, dispatches events
@Composable
fun UserScreen(vm: UserViewModel = hiltViewModel()) {
    val state by vm.uiState.collectAsStateWithLifecycle()
    when {
        state.loading -> CircularProgressIndicator()
        state.user != null -> UserCard(state.user!!) { vm.refresh() }
        state.error != null -> ErrorView(state.error!!) { vm.refresh() }
    }
}

// 4. Repository — interface in domain, impl in data
interface UserRepository {
    suspend fun getUser(id: String): User
    fun observeUser(id: String): Flow<User>
}

class UserRepositoryImpl @Inject constructor(
```

```
    private val api: UserApi,
    private val dao: UserDao
) : UserRepository {
    override suspend fun getUser(id: String): User {
        val remote = api.fetch(id)
        dao.upsert(remote)                          // cache
        return dao.get(id)                          // always read from Room (single source of truth)
    }
    override fun observeUser(id: String) = dao.observeFlow(id)
}

// 5. Data sources
@Dao
interface UserDao {
    @Upsert suspend fun upsert(user: User)
    @Query("SELECT * FROM User WHERE id = :id") suspend fun get(id: String): User
    @Query("SELECT * FROM User WHERE id = :id") fun observeFlow(id: String): Flow<User>
}

interface UserApi {
    @GET("users/{id}") suspend fun fetch(@Path("id") id: String): User
}
```

Pros and cons

PROS

- ViewModel survives configuration changes (built-in)
- No View interface boilerplate
- State is observable — UI reacts automatically
- Easy to test (StateFlow assertions in `runTest`)
- First-class Jetpack support (viewModelScope, SavedStateHandle, hiltViewModel)
- Compose was designed around this pattern
- Lower boilerplate than MVI

CONS

- Easy to leak state across multiple StateFlows that should be one
- "What's the current state?" can be ambiguous if you have many fields not bundled into one
- One-shot events (toast, navigate) need careful handling — SharedFlow + replay=0
- Can drift toward "MVC with extra steps" if ViewModel becomes a god class

Common mistakes / misconceptions

- **Putting Android Context in ViewModel** — leak. Use Application context only via Hilt's `@ApplicationContext`.
- **Using LiveData for one-shot events** — they replay to new observers, double-firing toasts. Use `SharedFlow(replay=0)` or "Event wrapper" pattern.
- **Calling Repository from the View directly** — defeats the whole point. UI talks to ViewModel only.
- **One ViewModel per Composable** — wasteful. ViewModel is per logical screen, not per leaf widget.

- **Putting business logic in the ViewModel that should be in a UseCase** — fine for small apps; pulls into Domain when you reuse logic across screens.
- **Multiple StateFlows** when one would do — synchronization nightmares. Combine into one `UiState`.

Where things fit in MVVM

COMPONENT	WHERE	WHY
Activity / Fragment / Composable	View — observes state, dispatches intents	UI logic only; no business rules
ViewModel	Presentation — holds StateFlow, exposes events	Survives rotation; lifecycle-aware
UseCase (optional)	Domain — single business operation	Reuse across screens; pure Kotlin
Repository	Data layer — single source of truth	Hides which data source supplies what
Room DAO	Inside Repository	Local persistence
Retrofit Api	Inside Repository	Remote data source
DataStore	Inside Repository (or its own SettingsRepository)	Preferences / typed config
WorkManager	Triggered from Repository or a UseCase	Background sync, retry, deferred uploads
ForegroundService	Started from Application or a Repository; exposes Flow back	Long-running visible work (music, GPS, broadcast)
ContentProvider	Wrap as a DataSource; expose suspend / Flow API	Don't expose ContentProvider to ViewModel directly
BroadcastReceiver	Triggered → notify Repository (which then updates state)	Bridge between system events and your state
Hilt / Koin / Manual DI	Composition root: Application class for singletons; `@HiltViewModel` for VMs	Wire dependencies without each class building its own

Package structure (single-module MVVM)

```
com.example.app/
├── App.kt                          // @HiltAndroidApp
├── di/                             // Hilt modules
│   ├── DataModule.kt
│   ├── NetworkModule.kt
│   └── RepositoryModule.kt
├── data/
│   ├── local/
│   │   ├── AppDatabase.kt          // Room DB + DAOs
│   │   ├── UserDao.kt
│   │   └── entity/
│   ├── remote/
│   │   ├── UserApi.kt              // Retrofit interface
│   │   └── dto/                    // network DTOs
│   ├── repository/
│   │   └── UserRepositoryImpl.kt
│   └── mapper/                     // DTO ↔ domain mapping
├── domain/
│   ├── model/
│   │   └── User.kt                 // pure Kotlin domain model
│   ├── repository/
│   │   └── UserRepository.kt       // interface
│   └── usecase/                    // optional
│       └── GetUserUseCase.kt
├── ui/
│   ├── theme/                      // Compose theme
│   ├── common/                     // reusable Composables
│   ├── user/
│   │   ├── UserScreen.kt
│   │   ├── UserViewModel.kt
│   │   └── UserUiState.kt
│   └── feed/
│       ├── FeedScreen.kt
│       ├── FeedViewModel.kt
│       └── FeedUiState.kt
└── MainActivity.kt                 // single activity hosting NavHost
```

8. MVI — Model-View-Intent (state-machine architecture)

Diagram

```
        ┌──────────────────────────────────────────┐
        │                                          │
        │                                          ▼
┌──────────────┐   Intent   ┌──────────────────────────────┐
│     VIEW     │──────────▶│             VIEWMODEL             │
│  renders     │            │                                  │
│  UiState     │            │  ┌────────────────────────┐      │
│              │◀──────────│  │  reduce(state, intent)     │      │
└──────────────┘   UiState  │  │       = newState           │      │
                            │  └────────────────────────┘      │
                            │                                  │
                            │  ┌────────────────────────┐      │
                            │  │ UiState (immutable)        │      │
                            │  │ MutableStateFlow<UiState>  │      │
                            │  └────────────────────────┘      │
                            │                                  │
                            │  side effects (one-shot events)  │
                            └──────────────────────────────────┘
                                       │
                                       ▼
                            ┌──────────────┐
                            │  Repository  │
                            └──────────────┘
```

Origin / inspiration

MVI is **Redux for Android**. Redux (2015) came from Elm Architecture (2012). Both are functional-programming-inspired: model your UI as a finite state machine, where the only way to change state is to emit a typed intent that's reduced into a new state. The thought process: *"In MVVM with many StateFlows, race conditions and missed transitions are easy. What if state changes were forced through ONE function — reduce(state, intent) → newState? Then every transition is explicit, time-travel debugging is possible, and the state machine catches missed cases at compile time."*

Pure functional roots: **state monads** in Haskell, **actor model** in Erlang. Same idea: isolate state mutation behind a controlled interface.

Sample code

```
// 1. UI state — single immutable object
data class UserUiState(
    val loading: Boolean = false,
    val user: User? = null,
    val error: String? = null
)

// 2. Intent — every possible user/system action
sealed interface UserIntent {
    data class Load(val id: String) : UserIntent
    data object Refresh : UserIntent
    data class UpdateName(val name: String) : UserIntent
    data object Save : UserIntent
}

// 3. SideEffect — one-shot events the View should react to (navigation, toast)
sealed interface UserEffect {
    data class Navigate(val route: String) : UserEffect
    data class ShowToast(val message: String) : UserEffect
}

// 4. ViewModel — single source of truth, single reducer
@HiltViewModel
class UserViewModel @Inject constructor(
    private val repo: UserRepository
) : ViewModel() {
    private val _state = MutableStateFlow(UserUiState())
    val state: StateFlow<UserUiState> = _state.asStateFlow()

    private val _effects = Channel<UserEffect>(Channel.BUFFERED)
    val effects: Flow<UserEffect> = _effects.receiveAsFlow()

    fun dispatch(intent: UserIntent) {
        // reduce synchronously, then handle async side effects
        _state.update { reduce(it, intent) }
        handleSideEffects(intent)
    }

    private fun reduce(s: UserUiState, i: UserIntent): UserUiState = when (i) {
        is UserIntent.Load        -> s.copy(loading = true, error = null)
        UserIntent.Refresh        -> s.copy(loading = true, error = null)
        is UserIntent.UpdateName  -> s.copy(user = s.user?.copy(name = i.name))
        UserIntent.Save           -> s.copy(loading = true)
    }

    private fun handleSideEffects(i: UserIntent) = viewModelScope.launch {
        when (i) {
            is UserIntent.Load -> runCatching { repo.getUser(i.id) }
                .onSuccess { _state.update { s -> s.copy(loading = false, user = it) } }
                .onFailure { _state.update { s -> s.copy(loading = false, error = it.message) } }
            UserIntent.Save -> { repo.save(_state.value.user!!); _effects.send(UserEffect.ShowToast("Saved"))
```

```
            else -> { // no async side effect }
        }
    }
}

// 5. View (Compose) — renders state, dispatches intents
@Composable
fun UserScreen(vm: UserViewModel = hiltViewModel(), nav: NavController) {
    val state by vm.state.collectAsStateWithLifecycle()

    LaunchedEffect(Unit) {
        vm.effects.collect { effect ->
            when (effect) {
                is UserEffect.Navigate  -> nav.navigate(effect.route)
                is UserEffect.ShowToast -> toast(effect.message)
            }
        }
    }

    Column {
        if (state.loading) CircularProgressIndicator()
        state.user?.let {
            TextField(it.name, onValueChange = { vm.dispatch(UserIntent.UpdateName(it)) })
            Button(onClick = { vm.dispatch(UserIntent.Save) }) { Text("Save") }
        }
        state.error?.let { Text(it) }
    }
}
```

Pros and cons

PROS

- State is single + immutable; impossible to have inconsistent UI
- Every transition is explicit and traceable (great for time-travel debugging)
- Reducer is a pure function — trivially testable
- Compile-time safety: sealed Intent forces handling of every action
- Side effects clearly separated from state changes
- Concurrent intents serialize through the reducer — no race conditions

CONS

- More boilerplate per screen (Intent + Effect + UiState + reducer)
- Overkill for simple screens (a 3-field profile screen doesn't need a state machine)
- "How do I emit an event from inside a side effect" question — needs Channel/SharedFlow
- Newcomers struggle with the rigid structure
- If reducer becomes huge, you're back to god-class problem (split per feature)

When MVI shines

- Complex async screens with many states (chat, video player, multi-step checkout)

- Screens with many concurrent inputs (typing + scrolling + network update + push)
- Teams that want strict state-machine guarantees
- Apps that need detailed analytics on user actions (every Intent = an event)

Common mistakes

- **Reducer doing async work** — reducer must be pure. Side effects go in a separate function called after reduce.
- **Mutating UiState** — UiState must be immutable (data class with vals). Always emit a new copy.
- **Single SharedFlow for state AND events** — separate them. State = StateFlow. Events = Channel/SharedFlow with replay=0.
- **One giant Intent sealed class for the whole app** — split per screen. Each screen has its own UiState, Intent, Effect.
- **Skipping Effects entirely** — then you're back to events-as-state problem (toast appears twice on rotation).

Where things fit in MVI

Same as MVVM (§7) for everything below the ViewModel — Repository, DataSources, WorkManager, Services, ContentProvider, DI. The difference is purely in how the ViewModel exposes state to the View.

Package structure (MVI per feature)

```
ui/user/
├── UserScreen.kt              // @Composable, observes state, dispatches intents
├── UserViewModel.kt           // reducer + side effect handler
├── UserUiState.kt             // data class
├── UserIntent.kt              // sealed interface
├── UserEffect.kt              // sealed interface (one-shot events)
└── UserReducer.kt             // pure function (optional separate file)
```

9. Clean Architecture — adding the Domain layer

Diagram

```
┌──────────────────────────────────────────────┐
│  PRESENTATION (UI layer)                      │
│  Composables, ViewModels, UiState, Intents    │
│  - knows about Android, Compose               │
└──────────────────────────────────────────────┘
                   │
                   │ depends on
                   ▼
┌──────────────────────────────────────────────┐
│  DOMAIN (business rules)                      │
│  UseCases, Domain Models, Repository INTERFACES │
│  - PURE Kotlin (no Android, Retrofit, Room)   │
└──────────────────▲───────────────────────────┘
                   │ implements
                   │
┌──────────────────│───────────────────────────┐
│  DATA (implementations)                       │
│  Repository IMPLs, DAOs, Apis, DTOs, Mappers  │
│  - knows about Room, Retrofit, DataStore      │
└──────────────────────────────────────────────┘
```

Origin / inspiration

Robert C. Martin's **Clean Architecture** (2012, formalized in his 2017 book). Generalizes earlier ideas: **Hexagonal Architecture / Ports and Adapters** (Alistair Cockburn 2005), **Onion Architecture** (Jeffrey Palermo 2008). Common theme: *"Business rules are the most stable part of an app — they shouldn't depend on UI or persistence details. Invert dependencies so the outer layers (UI, DB) depend on the inner layer (business), not vice versa."*

Sample UseCase

```kotlin
// Domain layer — pure Kotlin, no Android imports
class GetUserWithOrdersUseCase @Inject constructor(
    private val userRepo: UserRepository,
    private val orderRepo: OrderRepository
) {
    suspend operator fun invoke(userId: String): UserWithOrders = coroutineScope {
        val user = async { userRepo.getUser(userId) }
        val orders = async { orderRepo.getOrdersFor(userId) }
        UserWithOrders(user.await(), orders.await())
    }
}

// ViewModel uses the UseCase instead of multiple repos
@HiltViewModel
class ProfileViewModel @Inject constructor(
    private val getUserWithOrders: GetUserWithOrdersUseCase
) : ViewModel() {
    fun load(id: String) = viewModelScope.launch {
        val data = getUserWithOrders(id)
        ...
    }
}
```

When to add a Domain layer

ADD USECASES WHEN	SKIP USECASES WHEN
Same business operation reused across multiple screens	Single screen calls Repository in one place
Logic combines multiple Repositories	UseCase would be a 1-line wrapper around Repo
You want to share business rules across Android + iOS via KMP	Small CRUD app — overhead not worth it
Logic needs heavy unit testing in isolation from data sources	You can already mock the Repo for the same coverage

Pros and cons

PROS

- Business rules isolated from UI and data details
- Trivially testable on JVM
- Same UseCases reusable across screens / platforms (KMP)
- Forces clean dependency graph
- Onboarding: new devs find logic in obvious places

CONS

- Boilerplate — each operation gets a UseCase class
- Many UseCases are 1-line wrappers — pure ceremony
- Over-abstraction is a real risk
- Beginners see the layers as bureaucracy

10. Multi-module architecture

Diagram

```
:app                          (thin glue: NavHost, Application class, theme)
   │
   ├──:core
   │    ├──:network            (OkHttp, Retrofit setup, common DTOs)
   │    ├──:database           (Room DB, DAOs, type converters)
   │    ├──:design-system      (Composables, theme, colors, typography)
   │    ├──:common             (utilities, extensions)
   │    └──:domain             (shared domain models, base interfaces)
   │
   ├──:feature
   │    ├──:home               (HomeScreen + ViewModel + UseCases + Repo + Data)
   │    ├──:profile            (ProfileScreen + ...)
   │    ├──:settings
   │    └──:checkout
   │
   └──:test-utils              (shared fakes, test rules)
```

Why split into modules

BENEFIT	HOW IT WORKS
Faster builds	Gradle compiles modules in parallel; touching one module doesn't recompile others
Enforced architecture	One module can't reach into another's `internal` classes; only public API is exposed
Ownership clarity	Each module has clear ownership; team boundaries map to module boundaries
Dynamic feature delivery	Modules can be marked as `com.android.dynamic-feature`; downloaded on demand from Play
KMP-ready	Only modules that need to be shared go to KMP; the rest stay Android-only

Dependency rules

- **:app** depends on every feature
- **:feature/*** depends on **:core/*** only — never on other features
- **:core/*** modules are leaves — no upward dependencies
- Cross-feature navigation goes through a "navigator" interface defined in :core, implemented in :app

Common mistake

Over-modularizing: 50+ tiny modules → Gradle's **configuration phase** becomes the bottleneck. Sweet spot for typical apps: 10-30 modules.

11. KMP-ready architecture (sharing logic with iOS)

Diagram

```
┌──────────────────────────┐          ┌──────────────────────────┐
│   ANDROID app            │          │   iOS app                │
│  Compose UI              │          │  SwiftUI                 │
│  Hilt DI                 │          │  Manual / Koin DI        │
│  androidMain source set  │          │  iosMain source set      │
└──────────────┬───────────┘          └──────────────┬───────────┘
               │                                     │
               └─────────────────┬───────────────────┘
                                 ▼
                 ┌──────────────────────────┐
                 │    SHARED MODULE (KMP)   │
                 │                          │
                 │  commonMain:             │
                 │    Domain (UseCases)     │
                 │    Repository INTERFACES │
                 │    Repository IMPLS      │
                 │    Models                │
                 │    Ktor (HTTP)           │
                 │    SQLDelight (DB)       │
                 │    kotlinx.serialization │
                 │    kotlinx.coroutines    │
                 │                          │
                 │  androidMain:            │
                 │    Android-specific impls │
                 │  iosMain:                │
                 │    iOS-specific impls    │
                 └──────────────────────────┘
```

What's KMP-friendly per architecture

LAYER	KMP-SHARED?	NOTES
UI (View / Composables / SwiftUI)	Mostly per-platform	Compose Multiplatform on iOS works but native UI often preferred
ViewModel	Can be shared (in commonMain) — needs `moko-mvvm` or your own	Most projects keep VMs per-platform; share UseCases instead
UseCases	YES — pure Kotlin, ideal for sharing	The biggest KMP win
Repository interface + impl	YES (in commonMain)	Use Ktor + SQLDelight + kotlinx.serialization
Domain models	YES	Pure data classes, no platform deps
Network (HTTP)	YES via **Ktor**	Replaces Retrofit in shared code
Local DB	YES via **SQLDelight** (or Room KMP 2.7+)	SQL-first; generates type-safe queries
JSON	YES via **kotlinx.serialization**	Replaces Moshi/Gson
DI	YES via **Koin** (Hilt is Android-only)	For mixed projects: Hilt on Android side, Koin in shared
Coroutines / Flow	YES — kotlinx.coroutines is KMP	Full support including Flow and Channels
WorkManager / ForegroundService	NO — Android-specific	Trigger from shared, but the actual code stays Android
ContentProvider	NO — Android-only	Wrap behind a shared interface; impl in androidMain

Pattern: `expect / actual` for platform APIs

```kotlin
// commonMain/src/Storage.kt
expect class Storage {
    fun save(key: String, value: String)
    fun get(key: String): String?
}

// androidMain/src/Storage.kt
actual class Storage(val ctx: Context) {
    private val prefs = ctx.getSharedPreferences("app", Context.MODE_PRIVATE)
    actual fun save(key: String, value: String) { prefs.edit().putString(key, value).apply() }
    actual fun get(key: String) = prefs.getString(key, null)
}

// iosMain/src/Storage.kt
actual class Storage {
    actual fun save(key: String, value: String) {
        NSUserDefaults.standardUserDefaults.setObject(value, key)
    }
    actual fun get(key: String) = NSUserDefaults.standardUserDefaults.stringForKey(key)
}
```

12. Modern patterns — Voyager, Decompose, Circuit

LIBRARY	WHAT IT ADDS	WHEN USE
Voyager	Compose-first navigation + screen-level state container; tabs, bottom-sheets, stacks. Replaces Compose Navigation for some teams.	Compose-only apps wanting simpler navigation
Decompose	KMP-friendly state management with parent-child component trees, predictable lifecycle. Pairs with Compose / SwiftUI / Web.	KMP-shared UI logic (decoupled from UI framework)
Circuit (Slack)	Compose-first MVI-ish pattern: each screen has a Presenter (Composable!) + UI Composable + State + Event. Strong type-safety, automatic reset on navigation.	Teams who want MVI's guarantees with less boilerplate
Molecule	Cash App's library: write Presenters as `@Composable` functions producing StateFlow. Reuse Compose's snapshot system for state computation outside the UI.	Reactive Presenter logic without writing reducers by hand

Circuit example (high-level)

```
// Circuit's Presenter is itself a Composable!
class UserPresenter(val repo: UserRepository) : Presenter<UserState> {
    @Composable
    override fun present(): UserState {
        var user by remember { mutableStateOf<User?>(null) }
        LaunchedEffect(Unit) { user = repo.getUser() }
        return UserState(user) { event ->
            when (event) {
                is UserEvent.Refresh -> user = repo.getUser()
            }
        }
    }
}
```

13. Where things fit — the master matrix

Same components, slightly different placement across architectures. Reference table for "where do I put X?":

COMPONENT	MVP	MVVM	MVI	CLEAN (ADDED)
UI rendering	Activity (View)	Activity / Composable	Activity / Composable	Same as inner
UI state	Presenter fields	StateFlow<UiState> in ViewModel	Single immutable UiState in ViewModel	Same
Business orchestration	Presenter	ViewModel	ViewModel + Reducer	UseCase
Data fetching	Repository	Repository	Repository	Repository (impl); interface in Domain
Local cache	Room DAO via Repository	Same	Same	Same
Network	Retrofit/Ktor via Repository	Same	Same	Same
Preferences	DataStore via Repository	Same	Same	Same
Background work	WorkManager — enqueued from Presenter	Enqueued from ViewModel or Repository	Enqueued from side-effect handler or Repository	Enqueued from UseCase
Long-running visible work	ForegroundService started from Activity / Presenter	Started from Application or via a controller VM	Same as MVVM	Same
System events	BroadcastReceiver → Presenter	BroadcastReceiver → Repository → ViewModel observes Flow	Same as MVVM (then dispatch as Intent)	Same
ContentProvider integration	Wrap in DataSource → Repository → Presenter	Same → Repository → ViewModel	Same	Same
DI	Manual or Hilt with field injection in Activity	Hilt: `@HiltViewModel` for VMs, `@Module` for repos	Same as MVVM	Same; UseCases get `@Inject constructor`
One-shot events (toast, navigate)	Direct method on View	SharedFlow<Event> with replay=0	Sealed Effect class via Channel	Same

COMPONENT	MVP	MVVM	MVI	CLEAN (ADDED)
Navigation	Activity calls `startActivity` from View handler	Compose Nav from View; navigation events from VM via SharedFlow	Effect → handled in View	Same

14. Comparison — when to prefer which

	MVP	MVVM	MVI	CLEAN (+ MVVM/MVI)
Boilerplate	Medium (View interface)	Low	High (Intent + Effect + Reducer)	Higher (UseCases)
Survives rotation	No (manual state save)	Yes (built-in)	Yes (built-in)	Yes
Testability	Excellent	Excellent	Best (pure reducer)	Best
Reactive UI fit	Manual	Native (StateFlow + Compose)	Native	Native
Compose-friendly	Awkward	First-class	First-class	First-class
State machine guarantees	None	Weak	Strong (sealed Intents)	Strong (per-screen)
KMP-friendly	OK (Presenter is plain Kotlin)	Need wrappers for ViewModel	Excellent (reducer is pure Kotlin)	Excellent (UseCases are KMP)
Java codebase OK?	Yes	Awkward (Coroutines needed)	Awkward	OK with refactor
Best for	Legacy apps; Java codebases	Most modern Android apps	Complex async screens, large teams	Multi-platform, big enterprise apps

Decision rules

1. **Greenfield Compose app, small team?** → MVVM. Most efficient.
2. **Greenfield Compose app, complex screens (chat, video, multi-step forms)?** → MVI for those screens; MVVM for simple screens. Mix is fine.
3. **Targeting iOS too via KMP?** → Clean Architecture in shared module + your choice of MVVM/MVI per platform UI.
4. **Legacy Java codebase?** → MVP for incremental migration; introduce ViewModel slowly.
5. **Multi-team enterprise app?** → Multi-module + Clean Architecture + MVVM/MVI per feature module.

15. Starting a fresh project — step by step

For MVVM (default for new apps)

1. Create a single-module Android Studio project with Compose template.
2. Add dependencies: Hilt, Room, Retrofit, Moshi/kotlinx.serialization, Coroutines, ViewModel-Compose, Navigation-Compose, lifecycle-runtime-compose.
3. Set up Hilt: `@HiltAndroidApp` on Application; create `di/` package with NetworkModule, DatabaseModule, RepositoryModule.
4. Define your `data/` and `domain/` packages (see structure in §7).
5. Create one feature folder under `ui/` with: `Screen` (Composable), `ViewModel`, `UiState`.
6. Write a UseCase only when you have a reason — don't pre-build the abstraction.
7. Set up Compose Navigation with type-safe routes.
8. Add a small set of design-system Composables (`AppButton`, `AppText`) to keep visuals consistent.
9. Write your first ViewModel test using runTest + Turbine.
10. Ship a Baseline Profile early — perf compounds.

For MVI

1. Same scaffolding as MVVM (steps 1-4).
2. Per feature folder: add `UiState` (data class), `Intent` (sealed interface), `Effect` (sealed interface), `Reducer`.
3. ViewModel exposes `state: StateFlow<UiState>`, `effects: Flow<Effect>`, and a `dispatch(intent)` function.
4. Write a base `MviViewModel<State, Intent, Effect>` abstract class to avoid repeating boilerplate per screen.
5. Composable: `val state by vm.state.collectAsStateWithLifecycle()`; `LaunchedEffect` to collect `vm.effects`.
6. Test the reducer in isolation as a pure function — no coroutines required.

For multi-module

1. Start with one app module + a few core modules (network, database, design-system, common).
2. Extract a feature into a `:feature:foo` module when:
   - It has > ~5-10 files
   - It can be developed independently (no cross-feature dependencies)
   - You'd benefit from independent compilation in CI
3. Use Gradle Version Catalog (`libs.versions.toml`) for shared dependency versions.
4. Define each feature's public surface in `:feature:foo:api` if you want strict separation; the impl module depends on the api.

16. SOLID interview "why this not that" — drill-down examples

Q: "Why use a Repository instead of calling Retrofit directly from the ViewModel?"

- **SRP:** ViewModel becomes responsible for both UI state AND data sourcing — two reasons to change.
- **DIP:** ViewModel depends on a concrete library (Retrofit). Replacing it (e.g., Ktor) means rewriting all ViewModels.
- **OCP:** Adding a cache means modifying every ViewModel. With a Repository, you change one place.
- **Testing:** Faking Retrofit requires MockWebServer. Faking a Repository interface is one class.

Q: "Why interface for the Repository in domain, impl in data?"

- **DIP:** Domain (high-level policy) shouldn't depend on Data (low-level details). Putting the interface in Domain inverts the dependency arrow.
- **KMP:** If domain is shared, the interface can be in commonMain while the impl can be platform-specific.
- **Testing:** UseCase test injects a fake without pulling in Room/Retrofit.

Q: "Why prefer composition over inheritance?"

- **LSP problems** show up frequently with deep inheritance — a subclass changes parent behavior in a surprising way.
- **Compose Modifiers** are a textbook example: instead of `RoundedClickableBorderedView extends View`, you compose `Modifier.background().clip().border().clickable()`.
- Inheritance hard-codes the relationship at compile time. Composition lets you change behavior at runtime.

Q: "Why use sealed classes for UiState instead of multiple booleans?"

- **SRP:** A loading boolean + error boolean + data field forces every consumer to interpret the combination — and they can disagree about what "loading=true && error=null && data=null" means.
- **Type safety:** Sealed class forces exhaustive `when` — you can't forget the Error state.
- **State machine guarantees:** The set of valid states is finite and named.

Q: "Why inject the dispatcher instead of using `Dispatchers.IO` directly?"

- **DIP + testability:** Tests can inject a `StandardTestDispatcher` for virtual time and deterministic execution.
- Without injection, your test's coroutines run on the real thread pool — slow, non-deterministic, and you can't advance time.

Q: "Why hoist state in Compose?"

- **SRP:** Stateless Composable does one thing — render. State management is the caller's responsibility.
- **Reusability:** A stateless Composable is testable, previewable, and reusable in different contexts.
- **OCP:** Adding new state behavior (validation, persistence) is done at the call site without modifying the Composable.

Q: "Why use Flow instead of LiveData?"

- **SRP:** LiveData is Android-coupled. Flow is pure Kotlin → KMP-friendly, testable on JVM without Android.
- Flow has a richer operator set (debounce, flatMapLatest, combine, etc.).
- StateFlow + collectAsStateWithLifecycle gives the same lifecycle awareness as LiveData but composes better with coroutines.

★ Evergreen rules — how to evaluate any new architecture (the developer thought process)

Patterns come and go. Twitter explodes about a new architecture every 6 months. But the underlying principles that make architecture "good" are 50 years old and don't change. When the next "MVU" or "TCA-Android" or "FlowRedux" or "Effect-Action-State" lands, judge it against these — not against the hype.

The 12 timeless rules (these don't change, ever)

RULE	WHAT IT MEANS	WHY IT NEVER GOES OUT OF STYLE
Separation of concerns	One class / module / function does one thing	Lets you reason locally without holding the whole system in your head
Single source of truth	One canonical place per piece of state	Prevents drift, race conditions, "which copy is right?"
Unidirectional data flow	State flows down; events flow up	Predictable, debuggable; matches how humans reason about programs
Dependency inversion	High-level modules depend on abstractions, not low-level details	Lets you replace impls (real DB ↔ fake DB) without touching policy
Composition over inheritance	Build behavior by combining small pieces, not extending a class hierarchy	Inheritance hard-codes relationships at compile time; composition is changeable at runtime
Pure functions	Same input → same output, no hidden side effects	Trivially testable, parallelizable, cacheable
Immutability	State doesn't mutate in place; replaced wholesale	Eliminates whole classes of bugs (race, time-travel)
Smallest possible scope	Variables, dependencies, lifetimes — make them tiny	Smaller scope = fewer interactions = fewer bugs
Explicit over implicit	Name the dependency, name the side effect	Hidden behavior is the source of "spooky" bugs
Test seams at boundaries	Interface every layer crossing	Lets you test in isolation without dragging in the universe
Fail loudly, locally	Crash where the error originates, with enough context	Hidden errors propagate and become impossible to debug
Reversibility	Easy to undo a decision later	Architectural choices that lock you in are dangerous

These came from Smalltalk, C++ design, Erlang, Haskell, Domain-Driven Design — long before Android existed. **Every "good" Android architecture is a different framing of these rules.** When you see a pattern that "skips" or "transcends" them, be skeptical.

The 10-question checklist for any new architecture

A new pattern just dropped on Twitter. Before you adopt it, walk through these:

1. **What problem does it solve that I actually have?**
   - "It's the new hotness" → not a reason
   - "It scales the team better" → maybe (depends on team size)
   - "It catches bug class X at compile time" → strong reason if you've shipped X bugs
   - "It enables use case Y the old pattern struggles with" → strong reason if you have Y
2. **Does it follow the 12 timeless rules above?** Walk through each. Patterns that violate one need a very strong justification.
3. **What's the escape hatch?** All abstractions leak. What's your plan when the framework can't model your problem? Patterns without escape hatches become handcuffs at scale.
4. **What's the migration path?** Big-bang rewrites are almost always wrong. Can the new pattern coexist with existing code (per-screen / per-module migration)?
5. **What's the lock-in level?** Tied to a specific library, vendor, or community? "If this library disappears tomorrow, can I extract the architecture pattern?" Patterns that work with standard kotlinx + Jetpack > patterns that need a niche dependency.
6. **Who's using it in production?** Open-source isn't enough. Look for: apps with > 10M users running on it for > 1 year; multiple teams in different domains; clear postmortems of failures.
7. **What's the testing story?** A pattern without a clear testing approach is not production-ready. "How do I unit-test the presentation logic?" should have a one-line answer.
8. **What's the tooling story?** IDE support, autocompletion, navigation, refactoring, lint rules. Without tooling, every new dev re-learns by reading a 200-page book.
9. **What does it cost in code volume?** Boilerplate is real. If a typical screen needs 5 files, that's 50 files for 10 screens. Justified only if it prevents real bugs you've shipped.
10. **What's the net SOLID rating?** Where does the pattern strengthen each SOLID principle (§3, §16)? Where does it weaken any?

Common "new architecture" red flags

RED FLAG	WHAT IT REALLY MEANS
"Eliminates need for ViewModels"	Misses why ViewModels exist (config-change survival, framework integration)
"No DI needed"	Hides dependencies; just makes them implicit + global
"It's like Redux but simpler"	Often skips Redux's actual guarantees (time-travel, replay, devtools)
"Compose-only" pattern that bypasses ViewModel	Loses lifecycle integration; reinvents Jetpack
Heavy use of macros / annotation processing	Build complexity, debugging difficulty, IDE struggles
Author works at Big Co, no production case study	Not battle-tested; could be Big Co's internal experiment
All examples are TODO apps	The pattern fails on real-world complexity
"Everything is reactive"	Reactive everything = stack traces become useless; debugging gets harder
Marketing pages with zero discussion of trade-offs	Author hasn't shipped it at scale yet; doesn't know the pain points

The contrarian rule — when boring wins

> **Most production teams should pick** *MVVM + Hilt + Compose + Room + Retrofit + Coroutines/Flow + Single-Activity Navigation* — and stop.
>
> This boring stack:
>
> - Has 5+ years of production track record at scale
> - Is what Google's docs assume
> - Works for 90% of apps
> - New hires already know it
> - Has the deepest Stack Overflow / docs / tooling support
>
> Reach for fancier patterns only when you have a **specific reason** the boring stack can't solve.

How to learn ANY new architecture in 10 minutes

When a new architecture pattern lands, read its docs in this order. Once you can answer these 6 questions, you understand it — regardless of what it's called:

1. **What does the View talk to?** (Presenter? ViewModel? Component? Store?)
2. **What is the source of state?** (Mutable? Immutable? Single object? Multiple flows?)
3. **How do events get from View to logic?** (Method call? Intent? Action? Event? Reducer?)

4. **How does state get from logic to View?** (Observable? Callback? Subscription? Recomposition?)
5. **What's the lifecycle story?** (Survives rotation? Process death? When is it cleared?)
6. **What's the testing story?** (Unit-test the logic? Mock or fake what?)

Bonus question 7: *What does this pattern call the same thing that MVVM calls "ViewModel" and "UiState"?* Most patterns are MVVM with renamed parts.

The mental rule that beats all hype

> **"Do my colleagues understand this in 10 minutes? Can a new hire be productive in a week?"**
>
> If yes → adopt it.
> If no → the cleverness cost is too high. Stay boring.

17. Related patterns worth knowing

PATTERN	WHERE YOU SEE IT IN ANDROID
Repository	Standard data abstraction (DDD-inspired). Hides data sources behind a domain-friendly API.
UseCase / Interactor	One operation = one class. Keeps business rules testable and composable.
Observer	StateFlow, SharedFlow, LiveData, callbacks — UI subscribes to changes.
Strategy	OkHttp Interceptors, Compose Modifiers, sort/filter strategies. Pluggable behavior behind an interface.
Singleton	Hilt's `@Singleton`, Kotlin `object`. App-scoped components (DB, OkHttp client).
Factory	ViewModelProvider.Factory (pre-Hilt), RecyclerView.Adapter.onCreateViewHolder.
Builder	OkHttpClient.Builder, NotificationCompat.Builder, Retrofit.Builder.
Adapter	RecyclerView.Adapter — bridges data list to View recycling.
Decorator	OkHttp Interceptors wrap each other; Compose Modifiers wrap composition.
Chain of Responsibility	OkHttp Interceptor chain; Android touch event dispatch.
Proxy	Retrofit's dynamic proxy generates HTTP-calling implementations of your interfaces.
Template Method	Activity/Fragment lifecycle hooks; RecyclerView.Adapter's onCreate/onBind.
State Pattern	Sealed UiState, ATM/VendingMachine state machines.
Mediator	ViewModel mediates between View and Repository; NavController mediates between screens.
Memento	onSaveInstanceState / SavedStateHandle.
Facade	Repository hides multiple data sources; `Glide.with(...).load(...).into(...)`.
Flyweight	RecyclerView.RecycledViewPool, BitmapPool, Drawable.ConstantState.
Command	Runnable, Intent, WorkRequest — encapsulate an action as an object.

18. Common mistakes across all architectures

ARCHITECTURE MISTAKES

- Calling Repository from the View directly (skipping VM/Presenter)
- Putting Android Context in domain/data layer (breaks JVM testing)
- Exposing Room `@Entity` directly to the UI (couples UI to DB schema)
- Multiple Repository singletons reading the same data without coordination
- Using `GlobalScope` instead of viewModelScope
- One ViewModel for the entire app
- Mixing layers: a UseCase that reads a SharedPreferences directly
- Returning Flow<Result> from Room (rare correct case; usually a smell)

PATTERN MISCONCEPTIONS

- "MVVM means data binding" — no. MVVM = ViewModel as observable state holder. Data binding is one impl.
- "MVI requires Redux" — no. Many Android MVI implementations are just sealed Intent + reduce function.
- "Clean Architecture means UseCases everywhere" — no. UseCases are valuable when reused; over-using them is ceremony.
- "Multi-module makes builds faster always" — only true if dependency graph is shallow. Deep transitive deps can make it worse.
- "You must pick MVVM OR MVI for the whole app" — pick per-screen. Most apps mix.
- "The Domain layer must contain UseCases" — UseCases are optional. Just having domain models + Repository interfaces in Domain is also "Clean enough."

19. Package structuring — full templates

Single-module MVVM (small app)

```
com.example.app/
├── App.kt
├── MainActivity.kt
├── di/
│   ├── DatabaseModule.kt
│   ├── NetworkModule.kt
│   └── RepositoryModule.kt
├── data/
│   ├── local/      (Room — DAO, DB, entities)
│   ├── remote/     (Retrofit — API, DTOs)
│   └── repository/ (RepositoryImpl)
├── domain/
│   ├── model/      (pure Kotlin domain types)
│   ├── repository/ (interfaces)
│   └── usecase/    (optional — empty until needed)
├── ui/
│   ├── theme/
│   ├── common/     (shared Composables)
│   └── feature_x/
│       ├── XScreen.kt
│       ├── XViewModel.kt
│       └── XUiState.kt
└── util/
```

Single-module MVI (state-machine flavor)

```
// Same as MVVM, but per feature folder:
ui/feature_x/
├── XScreen.kt
├── XViewModel.kt
├── XUiState.kt
├── XIntent.kt          // sealed interface
├── XEffect.kt          // sealed interface (one-shot events)
└── XReducer.kt         // optional separate file for the pure reducer
```

Multi-module enterprise (mix of MVVM/MVI per feature)

```
:app                          (NavHost, Application, theme glue)
:core
  :core:common                (KMP-ready utilities)
  :core:design-system         (shared Composables, colors, typography)
  :core:network               (OkHttp, Retrofit, common interceptors)
  :core:database              (Room DB, DAOs, type converters)
  :core:domain                (shared domain models, base Result type)
  :core:test-utils            (fakes, test rules)
:feature
  :feature:home
    src/main/kotlin/.../feature/home/
      data/                   (Repo impl + data sources)
      domain/                 (UseCases, models)
      ui/                     (Screen + VM + State)
      di/                     (Hilt module)
  :feature:profile            (same shape)
  :feature:checkout           (same shape)
```

KMP-shared (Android + iOS)

```
:app:android                  (Android-specific UI + DI wiring)
:app:ios                      (iOS app — Xcode project consuming :shared)
:shared                       (KMP module)
  src/commonMain/kotlin/
    domain/
      model/
      repository/             (interfaces)
      usecase/
    data/
      remote/                 (Ktor)
      local/                  (SQLDelight)
      repository/             (impls)
    di/                       (Koin definitions)
  src/androidMain/kotlin/
    Storage.android.kt        (actual)
  src/iosMain/kotlin/
    Storage.ios.kt            (actual)
```

20. One-page recall card

The mental model

UI ↔ ViewModel/Presenter ↔ (UseCase) ↔ Repository ↔ Data sources. Dependencies point inward. State flows down, events flow up. Domain has zero Android imports.

Pick your pattern

- **MVP** — legacy Java codebase or learning vehicle
- **MVVM** — default for new Compose apps; lowest boilerplate
- **MVI** — complex async screens; state-machine guarantees
- **Clean** — when shared business logic across screens or platforms (KMP)
- **Multi-module** — when build times or team boundaries demand it

Where things go (universal)

- **Network/DB calls** → inside Repository; never in ViewModel
- **WorkManager** → enqueued from Repository or UseCase
- **ForegroundService** → started from Application or controller VM
- **ContentProvider** → wrapped behind a DataSource interface
- **BroadcastReceiver** → notify Repository → it updates state Flow
- **Hilt** → `@HiltAndroidApp` on App; `@HiltViewModel` on VMs; `@Module` for repos
- **One-shot events** → Channel or SharedFlow with replay=0; never StateFlow

SOLID in a sentence each

- **S** — one class, one reason to change
- **O** — extend via interfaces, don't modify existing code
- **L** — subtypes are drop-in replacements
- **I** — many small interfaces beat one fat one
- **D** — depend on abstractions, not concretions (DI lives here)

Compose-era best moves

- Single Activity + NavHost
- StateFlow + collectAsStateWithLifecycle
- Sealed UiState (one immutable object per screen)
- SharedFlow for events
- SavedStateHandle for process-death state
- Hilt for DI; Koin if going KMP
- kotlinx.collections.immutable for stable list parameters

Avoid

- Activity-only architecture
- Calling Repository from View
- Android Context in Domain layer
- Multiple Repositories holding the same data without coordination
- Exposing Room entities to UI
- One ViewModel for the whole app
- Many StateFlows when one combined UiState would do
- GlobalScope

Read top to bottom for understanding · skim §13 (where-things-fit matrix), §16 (SOLID interview answers), §20 (recall card) before any architecture interview. Pair with the **Top 210 Android Interview Questions**, **Coroutines Cheat Sheet**, and **Compose Cheat Sheet** in this repo for full Android coverage.